Faculty In charge : AARTHI M
Class No.: PRP231

Digital Footprint of JAVA Programs

TASK-I

School of Computer Science and Engineering (SCOPE) Vellore Institute of Technology Vellore.

main.cpp

Share | Run | Output

```
        case 4:
            if (s.isEmpty())
                cout << "Stack is empty.\n";
            else
                cout << "Stack is not empty.\n";
            break;
        case 5:
            if (s.isFull())
                cout << "Stack is full.\n";
            else
                cout << "Stack is not full.\n";
            break;
        case 6:
            s.display();
            break;
        case 0:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice. Please try again.\n";
        }
    } while (choice != 0);

    return 0;
}
```

```
Enter size of stack: 3

--- Stack Operations Menu ---
1. Push
2. Pop
3. Peek (Top Element)
4. Check if Stack is Empty
5. Check if Stack is Full
6. Display Stack
0. Exit
Enter your choice: 1
Enter value to push: 43
43 pushed to stack.

--- Stack Operations Menu ---
1. Push
2. Pop
3. Peek (Top Element)
4. Check if Stack is Empty
5. Check if Stack is Full
6. Display Stack
0. Exit
Enter your choice:
```

Programiz C++ Online Compiler

main.cpp

Share | Run | Output

```
        }
        arr[++top] = value;
        cout << value << " pushed to stack.\n";
    }

    void pop() {
        if (isEmpty()) {
            cout << "Stack Underflow! Cannot pop.\n";
            return;
        }
        cout << "Popped: " << arr[top--] << endl;
    }

    void peek() {
        if (isEmpty()) {
            cout << "Stack is empty.\n";
            return;
        }
        cout << "Top element is: " << arr[top] << endl;
    }

    bool isEmpty() {
        return top == -1;
    }
```

```
Enter size of stack: 3

--- Stack Operations Menu ---
1. Push
2. Pop
3. Peek (Top Element)
4. Check if Stack is Empty
5. Check if Stack is Full
6. Display Stack
0. Exit
Enter your choice: 1
Enter value to push: 43
43 pushed to stack.

--- Stack Operations Menu ---
1. Push
2. Pop
3. Peek (Top Element)
4. Check if Stack is Empty
5. Check if Stack is Full
6. Display Stack
0. Exit
Enter your choice:
```

main.cpp

Share | Run | Output

```

    bool isFull() {
        return top == capacity - 1;
    }

    void display() {
        if (isEmpty()) {
            cout << "Stack is empty.\n";
            return;
        }
        cout << "Stack elements: ";
        for (int i = 0; i <= top; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }
};

int main() {
    int size;
    cout << "Enter size of stack: ";
    cin >> size;

    Stack s(size);
    int choice, value;

```

```
Enter size of stack: 3

--- Stack Operations Menu ---
1. Push
2. Pop
3. Peek (Top Element)
4. Check if Stack is Empty
5. Check if Stack is Full
6. Display Stack
0. Exit
Enter your choice: 1
Enter value to push: 43
43 pushed to stack.

--- Stack Operations Menu ---
1. Push
2. Pop
3. Peek (Top Element)
4. Check if Stack is Empty
5. Check if Stack is Full
6. Display Stack
0. Exit
Enter your choice:
```

main.cpp

Share | Run | Output

```
75
76  do {
77      cout << "\n--- Stack Operations Menu ---\n";
78      cout << "1. Push\n";
79      cout << "2. Pop\n";
80      cout << "3. Peek (Top Element)\n";
81      cout << "4. Check if Stack is Empty\n";
82      cout << "5. Check if Stack is Full\n";
83      cout << "6. Display Stack\n";
84      cout << "0. Exit\n";
85      cout << "Enter your choice: ";
86      cin >> choice;
87
88      switch (choice) {
89      case 1:
90          cout << "Enter value to push: ";
91          cin >> value;
92          s.push(value);
93          break;
94      case 2:
95          s.pop();
96          break;
97      case 3:
98          s.peek();
99          break;
```

```
Enter size of stack: 3

--- Stack Operations Menu ---
1. Push
2. Pop
3. Peek (Top Element)
4. Check if Stack is Empty
5. Check if Stack is Full
6. Display Stack
0. Exit
Enter your choice: 1
Enter value to push: 43
43 pushed to stack.

--- Stack Operations Menu ---
1. Push
2. Pop
3. Peek (Top Element)
4. Check if Stack is Empty
5. Check if Stack is Full
6. Display Stack
0. Exit
Enter your choice:
```

main.cpp

Share | Run | Output

```
100         case 4:
101             if (s.isEmpty())
102                 cout << "Stack is empty.\n";
103             else
104                 cout << "Stack is not empty.\n";
105             break;
106         case 5:
107             if (s.isFull())
108                 cout << "Stack is full.\n";
109             else
110                 cout << "Stack is not full.\n";
111             break;
112         case 6:
113             s.display();
114             break;
115         case 0:
116             cout << "Exiting...\n";
117             break;
118         default:
119             cout << "Invalid choice. Please try again.\n";
120         }
121     } while (choice != 0);
122
123     return 0;
124 }
125
```

```
Enter size of stack: 3

--- Stack Operations Menu ---
1. Push
2. Pop
3. Peek (Top Element)
4. Check if Stack is Empty
5. Check if Stack is Full
6. Display Stack
0. Exit
Enter your choice: 1
Enter value to push: 43
43 pushed to stack.

--- Stack Operations Menu ---
1. Push
2. Pop
3. Peek (Top Element)
4. Check if Stack is Empty
5. Check if Stack is Full
6. Display Stack
0. Exit
Enter your choice:
```

Programiz C++ Online Compiler

main.cpp

Share | Run | Output

```
47          }
48          cout << "Front element is: " << arr[front] << endl;
49      }
50
51      bool isEmpty() {
52          return size == 0;
53      }
54
55      bool isFull() {
56          return size == capacity;
57      }
58
59      void display() {
60          if (isEmpty()) {
61              cout << "Queue is empty.\n";
62              return;
63          }
64          cout << "Queue elements: ";
65          for (int i = 0; i < size; i++) {
66              int index = (front + i) % capacity;
67              cout << arr[index] << " ";
68          }
69          cout << endl;
70      }
71  };
72
```

```
Enter size of queue: 3

--- Queue Operations Menu ---
1. Enqueue
2. Dequeue
3. Peek (Front Element)
4. Check if Queue is Empty
5. Check if Queue is Full
6. Display Queue
0. Exit
Enter your choice: 1
Enter value to enqueue: 34
34 enqueued to queue.

--- Queue Operations Menu ---
1. Enqueue
2. Dequeue
3. Peek (Front Element)
4. Check if Queue is Empty
5. Check if Queue is Full
6. Display Queue
0. Exit
Enter your choice:
=== Session Ended. Please Run the code again ===
```

Programiz C++ Online Compiler

main.cpp

Share | Run

```
        int index = (front + i) % capacity;
        cout << arr[index] << " ";
    }
    cout << endl;
}
};

int main() {
    int size;
    cout << "Enter size of queue: ";
    cin >> size;

    Queue q(size);
    int choice, value;

    do {
        cout << "\n--- Queue Operations Menu ---\n";
        cout << "1. Enqueue\n";
        cout << "2. Dequeue\n";
        cout << "3. Peek (Front Element)\n";
        cout << "4. Check if Queue is Empty\n";
        cout << "5. Check if Queue is Full\n";
        cout << "6. Display Queue\n";
        cout << "0. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;
```

Output

```
Enter size of queue: 3

--- Queue Operations Menu ---
1. Enqueue
2. Dequeue
3. Peek (Front Element)
4. Check if Queue is Empty
5. Check if Queue is Full
6. Display Queue
0. Exit
Enter your choice: 1
Enter value to enqueue: 34
34 enqueued to queue.

--- Queue Operations Menu ---
1. Enqueue
2. Dequeue
3. Peek (Front Element)
4. Check if Queue is Empty
5. Check if Queue is Full
6. Display Queue
0. Exit
Enter your choice:
=== Session Ended. Please Run the code again ===
```

Programiz C++ Online Compiler

Programiz PRO >

main.cpp

Share | Run

Output | Clear

```
    cout << "6. Display Queue\n";
    cout << "0. Exit\n";
    cout << "Enter your choice: ";
    cin >> choice;

    switch (choice) {
    case 1:
        cout << "Enter value to enqueue: ";
        cin >> value;
        q.enqueue(value);
        break;
    case 2:
        q.dequeue();
        break;
    case 3:
        q.peek();
        break;
    case 4:
        if (q.isEmpty())
            cout << "Queue is empty.\n";
        else
            cout << "Queue is not empty.\n";
        break;
    case 5:
        if (q.isFull())
            cout << "Queue is full.\n";
```

Output

```
Enter size of queue: 3

--- Queue Operations Menu ---
1. Enqueue
2. Dequeue
3. Peek (Front Element)
4. Check if Queue is Empty
5. Check if Queue is Full
6. Display Queue
0. Exit
Enter your choice: 1
Enter value to enqueue: 34
34 enqueued to queue.

--- Queue Operations Menu ---
1. Enqueue
2. Dequeue
3. Peek (Front Element)
4. Check if Queue is Empty
5. Check if Queue is Full
6. Display Queue
0. Exit
Enter your choice:
=== Session Ended. Please Run the code again ===
```

Programiz C++ Online Compiler

Prog

main.cpp

Share | Run

Output

```
    case 4:
        if (q.isEmpty())
            cout << "Queue is empty.\n";
        else
            cout << "Queue is not empty.\n";
        break;
    case 5:
        if (q.isFull())
            cout << "Queue is full.\n";
        else
            cout << "Queue is not full.\n";
        break;
    case 6:
        q.display();
        break;
    case 0:
        cout << "Exiting...\n";
        break;
    default:
        cout << "Invalid choice. Please try again.\n";
    }
} while (choice != 0);

return 0;
}
```

Output

```
Enter size of queue: 3

--- Queue Operations Menu ---
1. Enqueue
2. Dequeue
3. Peek (Front Element)
4. Check if Queue is Empty
5. Check if Queue is Full
6. Display Queue
0. Exit
Enter your choice: 1
Enter value to enqueue: 34
34 enqueued to queue.

--- Queue Operations Menu ---
1. Enqueue
2. Dequeue
3. Peek (Front Element)
4. Check if Queue is Empty
5. Check if Queue is Full
6. Display Queue
0. Exit
Enter your choice:
=== Session Ended. Please Run the code again ===
```

~~AAI~~ TASK-1

Aim: To implement a stack data structure using an array with the following operations:

- push(): Insert an element into the stack
- pop(): Remove the top element from the stack.
- peek(): Return the top element without removing it
- isEmpty(): Check whether the stack is empty.
- isFull(): Check whether the stack is full

Algorithm:

Start

① Initialize top = -1 and define MAX size of the stack

② To push(item):

a. If top == MAX-1, the Display 'Stack Overflow'.

b. Else, increment top by 1 and set stack[top] = item

To pop():

a. If top == -1, then Display 'Stack Underflow'.

b. Else, retrieve stack[top], the decrement top by 1.

To peek():

a. If top == -1, display 'Stack is Empty'

b. Else, display stack[top]

To isEmpty():

a. If top == -1, return True

b. Else, return False.

To isFull():

a. If top == MAX-1, return True.

b. Else, return False

BCE0440 14/8

Program

```
#include <iostream>
using namespace std;
int stack[MAX], top = -1;
void push(int val){
    if(top==MAX-1)
        cout<<"Stack Overflow\n";
    else
        stack[++top] = val;
}
void pop(){
    if(top==-1)
        cout<<"stack is empty\n"
    else
        cout<<"Popped"<<stack[top--]<<endl;
}
void peek(){
    if(top==-1)
        cout<<"Stack is empty"<<endl;
    else
        cout<<"Top:"<<stack[top]<<endl;
}
void display(){
    if(top==-1)
        cout<<"Stack is empty"<<endl;
    else
        cout<<"Stack:";
```

```
    for (int i = top; i >= 0; i--)
        cout << stack[i] << " ";
    cout << endl;
  }
}

int main() {
  int choice, val;
  do {
    cout << "\n1. Push, 2. Pop, 3 Peek, 4. isEmpty, 5.
     isFull, 0. Exit";
    cin >> choice;
    switch (choice) {
      case 1: cout << "Enter value";
      cin >> val; push(val); break;

      case 2: pop(); break;
      case 3: peek(); break;
      case 4: cout << (top == -1 ? "Empty\n" :
      "Not Empty\n");
        break
      case 5: cout << (top == MAX-1 ?
      "Full\n" : "Not Full\n"); break;
      case 6: display(); break;
      case 0: cout << "Exit\n"; break;
      default: cout << Invalid\n";
  }
}
```

AIM:

To implement a Queue using an array and perform the following basic operations:

- enqueue() → Insert an element into the queue.
- dequeue() → Remove an element from the queue.
- peek() → View the front element.
- isEmpty() → Check if the queue is empty
- isFull() → Check if the queue is full.

Algorithm:

Start

14/8 BCE0440

Initialize front = -1, rear = -1, and define MAX size for the queue.

To enqueue (item):

a. If rear == MAX-1, display "Queue Overflow"

b. Else, if front == -1, set front = 0.

c. increase rear by 1

d. Insert item at queue[rear].

To dequeue():

a. If front == -1 or front > rear,
display "Queue Underflow".

b. Else. display queue[front]

c. Increase front by 1.

To peak():

a. If front = -1 or front > rear,
display "Queue is Empty".

b. Else, display queue[front].

To isEmpty():

a. If front == -1 or front > rear,
return True.

b. Else, return False

To isFull():

a. If rear == MAX - 1, return True

b. Else, return False.

End.

Program 8

```
#include <iostream>
using namespace std;

class Queue {
        int arr[100], front = -1, rear = -1;

public:
        void enqueue(int x) {
                if(rear == 99)
                cout << "Queue Full!" << endl;
                else {
                        if (front == -1) front = 0;
                        arr[++rear] = x;

                }
        }
        void dequeue() {
                if(front == -1 || front > rear)
                cout << "Queue Empty!\n";
                else front++;

        }
        void display() {
                if(front == -1)
                        cout << "Queue Empty" << endl;

                else
```

```
        for (int i=front; i<rear ; i++)
            cout << arr[i] << "  " <<endl;

        }

    }
};
int main() {

        Queue q;
        int choice, val;
        while (true) {
          cout<< "\n1. Enqueue \n 2. Dequeue\n
          3. Display\n 4. Exit\n Enter choice: ";

          cin >> choice;

        switch (choice) {
            case 1:
              cout <<"Enter value: ";

              cin >> val;
              q.enqueue(val);
              break;
            case 2:
                q.dequeue();
                break;
```

```
        case 3:
            q.display();
            break;
        case 4:
            return 0;
        default:
            cout << "Invalid choice";
        }
    }
}
```